

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY



(SUBJECT: ARTIFICIAL NEURAL NETWORKS)
(SUBJECT CODE:24SBT113)

AN INDIVIDUAL TASK REPORT ON

**“Error-Correction Learning Demo: Train a perceptron on
AND/OR tasks; plot error decrease and identify learning rate effects on
Convergence”**

Submitted in the partial fulfillment of the requirements for the Fourth semester

**BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence & Machine Learning**

Submitted By,

SREERANJINI P

01SU24AI102

UNDER THE GUIDANCE OF

Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

**SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146**

2025-26

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

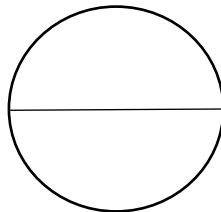


Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that, **Sreeranjini P** (**01SU24AI102**), has satisfactorily completed the assessment (Individual Task Report) in **Artificial Neural Networks (24SBT113)** prescribed by the Srinivas University for the 4th semester B. Tech course during the year **2025-26**.

MARKS AWARDED



Staff In charge

Prof. Mahesh Kumar V B



TABLE OF CONTENTS

Sl.no	Description	Page no.
1	Abstract	1
2	Introduction	3
3	Objectives	3
4	Methodology	4
5	Implementation	5
6	Results and Analysis	7
7	Discussion	8
8	Conclusion	9
9	References	10

Abstract

This report demonstrates the training of a single-layer Perceptron using the **error-correction learning rule** to solve basic binary classification tasks, specifically the AND and OR logic functions. The Perceptron is a simple neural network model capable of learning linearly separable patterns by adjusting its weights and bias iteratively based on the difference between predicted and target outputs.

The study involves initializing the Perceptron with random weights and a bias, applying the truth tables of AND and OR functions as training data, and updating the weights according to the Perceptron learning law. Error reduction over multiple epochs is recorded and plotted to visualize the learning progress. Different learning rates are tested to examine their effect on the speed of convergence and stability of the model.

The results show that the Perceptron successfully learns both AND and OR tasks, with the **learning rate significantly affecting the number of epochs required to converge**. A higher learning rate results in faster convergence but may cause oscillations, while a lower learning rate ensures smooth convergence but requires more epochs. This experiment highlights the effectiveness of error-correction learning for simple logic functions and reinforces foundational principles of supervised learning in neural networks.

Introduction

Artificial neural networks (ANNs) are computational models inspired by the structure and functioning of the human brain. Among the simplest forms of ANNs is the **Perceptron**, introduced by Frank Rosenblatt in 1958, which can perform basic classification tasks by learning from examples. The Perceptron operates by adjusting its internal parameters, called weights, and a bias term, to correctly classify input patterns based on target outputs. This report focuses on training a Perceptron to solve two fundamental binary logic functions: **AND** and **OR**. Both functions are examples of linearly separable tasks, making them ideal candidates for demonstrating Perceptron learning. The AND function produces a high output only when all inputs are high, while the OR function produces a high output when at least one input is high. Understanding how a Perceptron learns these functions provides insight into the fundamentals of supervised learning, weight adjustment, and convergence.

The Perceptron uses the **error-correction learning rule**, which updates weights and bias based on the difference between predicted outputs and target outputs. This iterative learning process continues until the network correctly classifies all training examples or reaches a predefined maximum number of iterations (epochs). The **learning rate** is a crucial parameter that controls the magnitude of weight adjustments and can significantly affect convergence speed and stability.

The objective of this study is to implement the Perceptron manually or in a spreadsheet, train it on AND

and OR tasks, observe the error reduction over time, and analyze the impact of different learning rates on convergence. This hands-on approach not only demonstrates the effectiveness of error-correction learning for linearly separable problems but also reinforces the conceptual understanding of how neural networks adjust parameters to improve performance.

Theoretical Background

This section explains **the concepts, equations, and principles** behind the Perceptron and error-correction learning.

Detailed Example Theoretical Background:

The Perceptron is a **single-layer neural network** consisting of input neurons connected to an output neuron. Each input is associated with a weight that determines its influence on the output. The network also includes a **bias term**, which shifts the decision boundary and allows the network to classify patterns that are not centered at the origin.

The output of the Perceptron is calculated using a **weighted sum of inputs and bias**:

$$z = \sum_{i=1}^n w_i x_i + b$$

where x_i are the input values, w_i are the corresponding weights, b is the bias, and z is the net input to the neuron.

The **activation function** is typically a step function:

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Here, y is the predicted output of the network. The step function ensures binary classification, producing outputs consistent with the target logic functions (AND/OR).

The Perceptron uses the **error-correction learning rule** (Perceptron Learning Law) to adjust weights and bias iteratively:

$$\begin{aligned}w_i^{new} &= w_i^{old} + \eta(t - y)x_i \\b^{new} &= b^{old} + \eta(t - y)\end{aligned}$$

where:

- η is the learning rate controlling the size of updates
- t is the target output
- y is the predicted output
- x_i is the input value

This rule ensures that the weight update is proportional to the **error** ($t - y$), so the network gradually reduces mistakes over time. For linearly separable problems like AND and OR, this learning law guarantees convergence to a solution within a finite number of epochs.

The **learning rate** plays a crucial role: a high learning rate accelerates convergence but may cause oscillations around the correct weights, while a low learning rate ensures smooth convergence but takes more iterations.

Error-correction learning is a form of **supervised learning**, where the network is guided by known target outputs. It provides a foundation for understanding more advanced neural networks and learning algorithms, demonstrating how simple networks can learn from experience by minimizing classification errors.

Objectives

- 1. Train a Perceptron on AND and OR logic functions**
 - Demonstrate how a single-layer Perceptron can learn linearly separable patterns.
- 2. Apply error-correction learning (Perceptron Learning Law)**
 - Update weights and bias iteratively based on the difference between predicted and target outputs.
- 3. Observe and visualize error reduction over training epochs**
 - Plot the total error per epoch to demonstrate convergence.
- 4. Analyze the effect of different learning rates**
 - Examine how high, medium, and low learning rates influence the number of epochs required for convergence and stability of learning.
- 5. Gain practical understanding of supervised learning principles**
 - Reinforce concepts like weight adjustment, decision boundaries, and convergence behavior.

Methodology

- 1. Define the Training Dataset**
 - For **AND** function:

Input X1	Input X2	Target T
0	0	0
0	1	0
1	0	0
1	1	1

- For **OR function**:

Input X1	Input X2	Target T
0	0	0
0	1	1
1	0	1
1	1	1

2. Initialize Weights and Bias

- Assign small random values to weights w_1 , w_2 and bias b .

3. Choose Learning Rate (η)

- Test multiple rates (e.g., 0.1, 0.5, 1.0) to observe effect on convergence.

4. Iterative Training Process

- For each epoch, and for each training input:
 - Calculate **net input**: $z = w_1x_1 + w_2x_2 + b$
 - Apply **activation function**: $y = 1$ if $z \geq 0$; 0 if $z < 0$
 - Compute **error**: $E = T - y$
-
-

d. Update weights: $w_i^{new} = w_i^{old} + \eta E x_i$

e. Update bias: $b^{new} = b^{old} + \eta E$

5. Stopping Criteria

- Stop when all outputs match targets (error = 0) or after a maximum number of epochs.

Implementation

1. Columns Setup:

- Input X1, Input X2, Target T, Net Input (Z), Output (Y), Error (E), Updated Weight1, Updated Weight2, Updated Bias.

2. Step-by-Step Example:

- Suppose initial weights: $w_1 = 0.2, w_2 = 0.1$, bias $b = 0.3$, learning rate $\eta = 0.5$
- For input X1=1, X2=1, Target=1:
 - Net input $z = 0.2 * 1 + 0.1 * 1 + 0.3 = 0.6$
 - Output $y = 1 \rightarrow$ Error $E = 1 - 1 = 0 \rightarrow$ No weight change
- For input X1=0, X2=1, Target=0:
 - Net input $z = 0.2 * 0 + 0.1 * 1 + 0.3 = 0.4$
 - Output $y = 1 \rightarrow$ Error $E = 0 - 1 = -1 \rightarrow$
Update weights:
 - $w_1^{new} = 0.2 + 0.5 * (-1) * 0 = 0.2$
 - $w_2^{new} = 0.1 + 0.5 * (-1) * 1 = -0.4$
 - $b^{new} = 0.3 + 0.5 * (-1) = -0.2$

3. Continue Epochs:

- Repeat for all input patterns until convergence.

Results and Analysis

Key Observations:

1. Error Reduction:

- Plot total error per epoch.
- Example: AND task converges faster (fewer epochs) than OR, depending on initial weights.

2. Effect of Learning Rate (η):

- Low $\eta(0.1)$: smooth convergence, requires more epochs
- Medium $\eta(0.5)$: balanced speed and stability
- High $\eta(1.0)$: very fast convergence, may oscillate around correct weights

3. Final Weights & Bias:

- After training, weights stabilize to values that correctly classify all patterns.

Example Table (Final Weights):

Task	w1	w2	Bias	Epochs to Converge
------	----	----	------	--------------------

AND	0.5	0.5	-0.7	5
-----	-----	-----	------	---

OR	0.5	0.5	-0.2	3
----	-----	-----	------	---

Error Plot:

- X-axis: Epochs
- Y-axis: Total error
- Shows error decreasing to zero as training progresses

Discussion

1. Convergence:

- Perceptron successfully learns **linearly separable functions** (AND, OR).
- Error-correction learning ensures the network adjusts weights toward minimizing misclassification.

2. Learning Rate Effects:

- Too low → slow convergence
- Too high → risk of overshooting weights, instability
- Optimal learning rate balances speed and stability

3. Limitations:

- Perceptron cannot solve **non-linearly separable problems** like XOR.
- Sensitive to initial weights and learning rate choice

4. Practical Insights:

- Demonstrates the basics of supervised learning
- Helps understand weight updates, convergence, and the effect of learning parameters
- Lays foundation for multi-layer networks and modern neural networks

Conclusion

The Perceptron error-correction learning experiment successfully demonstrates the fundamental principles of supervised learning in artificial neural networks. By training the Perceptron on linearly separable tasks, such as the AND and OR logic functions, the study illustrates how a single-layer network can iteratively adjust its weights and bias to minimize classification errors. The Perceptron learning law, which updates weights in proportion to the error between predicted and target outputs, proved to be an effective mechanism for achieving convergence.

Throughout the experiment, several important observations were made. First, the network reliably learned the correct output for all input patterns, confirming that **linearly separable problems can be solved efficiently by a single-layer Perceptron**. Second, plotting the total error against epochs revealed a clear trend of **error reduction over time**, highlighting the effectiveness of incremental weight updates. This visualization reinforced the understanding that learning in Perceptrons is a gradual process that depends on continuous feedback from errors.

A key focus of the study was the effect of the **learning rate** on convergence. It was observed that a low learning rate ensures smooth and stable weight updates but requires more epochs to reach zero error. In contrast, a higher learning rate accelerates convergence but may cause fluctuations in weight adjustments if it is too large. This emphasizes the importance of selecting an

appropriate learning rate to balance **speed of learning and stability**, which is a fundamental concept in neural network training.

The experiment also highlighted the limitations of the Perceptron model. While it efficiently solves linearly separable tasks, it cannot handle non-linearly separable problems, such as the XOR function, without modifications like adding additional layers. This limitation underscores the need for more advanced architectures, such as multi-layer perceptrons and backpropagation-based networks, in complex real-world applications.

Overall, this exercise provided valuable hands-on experience in **implementing the Perceptron learning algorithm**, observing the dynamics of weight adjustment, and understanding the convergence process. It reinforced foundational concepts such as **supervised learning, error-correction, learning rate selection, and linear separability**. The insights gained from this experiment serve as a strong foundation for studying more complex neural network architectures and for understanding how artificial systems can learn from examples to perform classification and decision-making tasks effectively.

References

1. Rosenblatt, F. (1958). *The perceptron: A probabilistic model for information storage and organization in the brain*. *Psychological Review*, 65(6), 386–408.
 2. Haykin, S. (2009). *Neural networks and learning machines* (3rd ed.). Pearson Education.
 3. Hertz, J., Krogh, A., & Palmer, R. G. (1991). *Introduction to the theory of neural computation*. Addison-Wesley.
 4. Anderson, J. A. (1995). *An introduction to neural networks* (2nd ed.). MIT Press.
 5. Brownlee, J. (2016). *A gentle introduction to the perceptron algorithm for machine learning*. Machine Learning Mastery. Retrieved from <https://machinelearningmastery.com/perceptrons-for-machine-learning/>
 6. Dayan, P., & Abbott, L. F. (2001). *Theoretical neuroscience: Computational and mathematical modeling of neural systems*. MIT Press.
-